

CSC 212: Data Structures and Abstractions

12: Analysis of Recursive Algorithms

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Spring 2026



Types of recursion

• Linear recursion

- ✓ at most one recursive call per activation
- ✓ call tree is a single path; depth proportional to n
- ✓ e.g. recursive factorial

Tail recursion: recursive call is the very last operation, no pending computation after the call returns (compilers may optimize into a loop)

• Binary recursion

- ✓ exactly two recursive calls per activation
- ✓ call tree is binary; can lead to exponential cost due to redundant computation of overlapping subproblems
- ✓ e.g. recursive fibonacci

Mutual recursion: two or more functions call each other, no single function calls itself; recursion arises from the cycle

• Multiple recursion

- ✓ more than two recursive calls per activation
- ✓ e.g., generating permutations, n -ary tree traversal

2

Recurrence relations

Recurrences

• Recurrence relation (a.k.a. recurrence)

- ✓ equation that expresses the terms of a sequence (beyond the initial conditions) as a function of one or more preceding terms
- ✓ base cases must be specified to uniquely define the sequence
- ✓ e.g., $T(n) = T(n - 1) + 1$

• Recurrences and algorithm analysis

- ✓ recurrences are used to analyze the time complexity of recursive algorithms
 - the time complexity $T(n)$ of a recursive algorithm can be expressed by a recurrence relation
 - solving a recurrence means finding a closed-form formula for $T(n)$
 - an exact closed-form solution may not exist or may be difficult to derive
- ✓ for most recurrences, an asymptotic solution of the form $\Theta(f(n))$ is typically sufficient

4

Techniques for solving recurrences

• recursion tree

- ✓ draw a recursion tree and sum the costs at each level (not a formal proof)

• iteration

- ✓ iteratively expand (unroll) the recurrence until a pattern emerges, then express the general case and apply the base case
- ✓ not trivial in all cases but it is helpful to build an intuition
- ✓ induction may be necessary to prove correctness

• substitution

- ✓ guess the form of the solution and prove it works by induction

• master theorem

- ✓ a shortcut for solving recurrences of the form $T(n) = aT(n/b) + f(n)$

5

Practice

$$T(0) = 0$$

$$T(n) = T(n - 1) + 1$$

```
double power(double b, int n) {  
    if (n == 0) {  
        return 1;  
    }  
    return b * power(b, n-1);  
}
```



6

Practice

$$T(0) = 0$$

$$T(1) = 0$$

$$T(n) = T(n - 2) + 1$$

7

Practice

$$T(0) = 0$$

$$T(n) = T(n - 1) + n$$

8

Practice

$$T(0) = 1$$

$$T(n) = 2T(n - 1) + 1$$

9

Practice

$$T(1) = 1$$

$$T(n) = 2T\left(\frac{n}{2}\right) + 1$$

10

Binary search

Binary search

• Search algorithm for sorted sequences

- ✓ **binary search** is an **efficient** algorithm for locating a **target value** within a sorted array
- ✓ the ordered nature of the data is exploited to achieve **logarithmic** time complexity

• Algorithm (recursive formulation)

- ✓ **base case:** if the input sequence is empty, the target is not present, return false
- ✓ compare the target value to the middle element of the input sequence
- ✓ if the target equals the middle element, return true (or the index of the element found)
- ✓ **recursive cases:** if the target differs from the middle element:
 - if target value is less than the middle element, recursively search the left half
 - if target value is greater than the middle element, recursively search the right half

12

Binary search

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

lo

hi

k = 48?

13

Binary search

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

lo

mid

hi

k = 48?

14

Binary search

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

lo

hi

k = 48?

15

Binary search

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

lo

mid

hi

k = 48?

16

Binary search



0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

lo mid hi

k = 48?

17

Show me the code

```
bool bsearch(std::vector<int>& A, size_t lo, size_t hi, int k) {
    if (hi < lo) {
        return false;
    }

    size_t mid = lo + ((hi-lo)/2); // safer than (lo+hi) / 2

    if (A[mid] == k) {
        return true;
    } else if (A[mid] < k) {
        return bsearch(A, mid+1, hi, k);
    } else {
        if (mid == 0) return false; // prevent size_t wraparound
        return bsearch(A, lo, mid-1, k);
    }
}
```

18

Practice

- Draw the recursion call tree (k=48)

0	1	2	3	4	5	6	7	8	9	10	11	12
1	2	5	10	15	20	22	30	35	40	43	48	51

- What is the complexity?

✓ best case? worst case? average case?

```
bool bsearch(std::vector<int>& A, size_t lo, size_t hi, int k) {
    if (hi < lo) {
        return false;
    }
    size_t mid = lo + ((hi-lo)/2); // safer than (lo+hi) / 2
    if (A[mid] == k) {
        return true;
    } else if (A[mid] < k) {
        return bsearch(A, mid+1, hi, k);
    } else {
        if (mid == 0) return false; // prevent size_t wraparound
        return bsearch(A, lo, mid-1, k);
    }
}
```

19

Analysis of binary search

$$T(0) = 0$$

$$T(n) = c + T(n/2)$$

```
bool bsearch(std::vector<int>& A, size_t lo, size_t hi, int k) {
    if (hi < lo) {
        return false;
    }
    size_t mid = lo + ((hi-lo)/2); // safer than (lo+hi) / 2
    if (A[mid] == k) {
        return true;
    } else if (A[mid] < k) {
        return bsearch(A, mid+1, hi, k);
    } else {
        if (mid == 0) return false; // prevent size_t wraparound
        return bsearch(A, lo, mid-1, k);
    }
}
```

count the number of comparisons

20